

求职投递话术卡 + 完整面试模拟

AI Agent / LLM 应用开发 · 实习岗

黄浩云 · 岭南师范学院 · 人工智能 · 2023-2027

本文件分两部分：第一部分是投递阶段的话术卡（打招呼、邮件自荐信、定向微调、投递提示）；第二部分是一次连贯的完整面试模拟（对话体剧本），从头到尾走一遍真实面试流程，练熟即可上场。

第一部分 · 投递话术卡

说明：以下是可直接复制使用的话术。核心策略是——用「已上线项目 + 独立完成」作为差异化卖点，30 秒内让 HR 记住你。

1.1 聊天窗口打招呼（BOSS 直聘 / 脉脉 / 官网私信）

一句话版（海投通用）

您好，我是岭南师范学院人工智能专业大三学生，独立完成过 3 个项目——一个已上线的浏览器 AI 扩展、一个 GraphRAG 检索平台、一个 Multi-Agent 面试系统，求职 LLM / Agent 应用开发实习，周内可到岗、可长期实习。作品集 me.jianla.xyz，期待进一步沟通。

两句版（岗位匹配时用）

您好，看到贵司在招 LLM 应用开发实习，我的背景比较对口：独立用 FastAPI 做过 GraphRAG 检索平台（Milvus + Elasticsearch + Neo4j 三路混合检索）和一个 Multi-Agent 面试系统（4 个 Agent + Orchestrator 编排）。

另外还有一个浏览器 AI 扩展已通过 Chrome/Edge 商店审核正式上线。可长期实习、周内到岗，简历和作品集可以发您吗？

1.2 邮件自荐信（标准版，可改）

主题

应聘 AI Agent / LLM 应用开发实习 —— 岭南师范学院 大三 黄浩云

正文

您好：

我是岭南师范学院人工智能专业大三学生，求职 AI Agent / LLM 应用开发方向实习，周内可到岗、可长期实习。

我的优势是三个项目都是独立完成、跑通了从开发到部署的完整闭环，不是课程作业：

1. 鉴来助手（已上线）—— 长文本结构化蒸馏的 AI 阅读助手，Chrome/Edge 商店审核通过、完成 ICP/公安备案，多渠道分发使用中；
2. GraphMind —— 工程化 GraphRAG 平台，Milvus + Elasticsearch + Neo4j 三路混合检索 + RRF 融合，带 Golden Eval 质量评测；
3. AI Interview Agent —— Multi-Agent 面试训练平台，5 阶段递进面试 + 4 维自动评分，149 后端 + 114 前端测试，CI/CD 容器化部署。

技术栈覆盖 FastAPI、SSE、SQLite、Docker、Nginx，熟悉 RAG / Agent 编排。

作品集（含界面截图与量化数据）：me.jianla.xyz GitHub: github.com/haoyunh415-create

附件是我的简历，期待有机会进一步沟通，谢谢！

黄浩云 18826604241 2313370765@qq.com

1.3 岗位定向微调（只换「匹配点」那两句）

投 RAG / 检索岗位时

我重点做过检索增强方向：自研 GraphRAG 平台，打通「文档入库 → 三路混合检索（向量 + BM25 + 图谱）→ 引用裁剪 → 答案生成 → 质量评测」闭环，并做了确定性的 Golden Eval 回归，每次改动都能量化对比效果。对 RAG 的工程落地有完整实践。

投 Agent / 多智能体岗位时

我重点做过 Agent 编排：把面试流程拆成 4 个专业 Agent（简历解析 / 面试官 / 评分 / 报告），由 Orchestrator 统一调度，通过 SharedMemory + MessageBus 松耦合通信，做到了各 Agent 可独立替换、独立测试。对 Multi-Agent 架构有真实落地经验。

1.4 投递实用提示

- 文件命名：黄浩云-AI 应用开发实习-可长期实习-周内到岗.pdf（别用「黄浩云简历」裸名）
- 投递时间：工作日早上 9:30-11:00，HR 处理完晨会，回复率最高
- 邮件正文别只甩附件，把自荐信里「三个项目」那段保留——差异化就靠它
- 作品集链接 me.jianla.xyz 放正文第一屏，别埋在结尾

第二部分 · 完整面试模拟（对话体剧本）

说明：下面是一次完整面试的连贯模拟——面试官连续提问、你连续作答。面试官问题加粗（铜色），你的回答是「可以直接说出口的示例表达」，练的时候先自己答，再对照示例。全程约 40~50 分钟，与真实实习面节奏一致。

角色设定

角色	设定
面试官	某公司 AI 应用开发团队技术负责人
候选人	黄浩云（岭南师范学院 · 人工智能 · 大三）
岗位	AI Agent / LLM 应用开发实习
时长	约 40~50 分钟

— 阶段 0 · 开场寒暄 —

面试官 你好，黄浩云，欢迎来面试。今天主要想聊聊你的项目和基础，放轻松。先简单介绍一下你自己吧。

我 面试官你好，我叫黄浩云，是岭南师范学院人工智能专业的大三学生，找 AI 应用方向的实习，可长期实习、周内到岗。

我 我独立做过三个 AI 项目，最有代表性的是「鉴来助手」——一个已经上架 Chrome 和 Edge 双商店的浏览器扩展，用渐进式摘要和伏笔雷达解决长篇小说的记忆断层问题，从过审、ICP 备案到分发渠道都是我一个人完成的。另外我还做了一个 Graph RAG 知识平台 GraphMind，融合向量、BM25 和图谱三路检索；以及一个多智能体协作的面试训练系统。

我 我的技术栈主要是 Python + FastAPI，熟悉 RAG、向量检索、多智能体和 Linux 部署。希望能把独立上线产品的经验带进团队。谢谢。

— 阶段 1 · 项目深挖（鉴来助手） —

面试官 你简历上最亮眼的是鉴来助手，能讲讲它到底解决了什么问题吗？

我 长篇小说连载几百章，读者追更几个月，早期的伏笔和人物关系早就记不清了。这不是简单的「忘记」，而是超长文本的结构性问题——和长会议纪要、知识库检索其实是同源的。我用「按章结构化蒸馏」把几百章压缩成人物、伏笔、术语这些结构化记忆，读者需要细节时再回看原文。

面试官 为什么用「渐进式摘要」而不是一次性把整本小说丢给 RAG？

我 一次性 RAG 在超长文本下有两个问题：一是检索漂移，整本书向量化后相关块容易漏；二是上下文成本，全塞进去又贵又慢。渐进式摘要的思路是分层压缩——章节先蒸馏成结构化要素，再按批次

生成阶段总结，最后汇总。这样在「召回质量」和「成本」之间做了平衡，需要细节时还能定位回原文。

面试官 伏笔雷达具体是怎么识别跨章伏笔的？

我 在渐进式摘要产出的结构化记忆之上做跨章呼应：把早期章节埋下的伏笔要素和后期回收的情节做关联匹配，输出伏笔列表。核心是先有「结构化记忆」这层底座，跨章匹配才有据可依。

面试官 Chrome 商店审核是一遍过的吗？

我 不是，一开始被打回了。原因是权限声明太宽，host_permissions 用了通配符，审核质疑。我把它收窄到单域名，逐条填写权限用途才过审。核心是「功能覆盖」和「审核风险」的权衡——收敛权限换取过审和用户信任。

面试官 你的部署架构是什么样的？

我 阿里云 Ubuntu + Nginx 反向代理到 uvicorn，Let's Encrypt 的 SSL 证书，systemd 守护进程，SQLite 存用户。FastAPI 后端调 DeepSeek 的 API 做 AI 分析。

面试官 国内用户装不了 Chrome 商店，你怎么解决分发？

我 提供免梯子的 zip 包，另外做了油猴脚本版，覆盖 Tampermonkey、Via、Alook 这些国内常用的脚本管理器。这是从「国内用户真实约束」倒推出来的必选项，不是可选项。

面试官 项目上线后用户量怎么样？你怎么看这个结果？

我 说实话用户量不多，个位数，我还在诊断原因。可能的原因有几个：目标用户是网文读者、对浏览器扩展的认知门槛比较高，付费意愿和获客渠道也需要验证。这是我后面要重点补的一课——不能只埋头开发，要更早做用户验证。

— 阶段 2 · 项目深挖 (GraphMind) —

面试官 再讲讲 GraphMind。为什么要做「图检索」，纯向量检索的短板是什么？

我 纯向量检索擅长语义相似，但回答不了「跨文档的实体关系、多跳推理」这类问题，比如「A 和 C 通过 B 有什么关系」。知识图谱把知识抽成实体和关系存进 Neo4j，检索时能做多跳遍历，补足了关系建模。这是 GraphMind 的核心动机。

面试官 三路检索具体是怎么融合的？

我 向量检索 (Milvus) 负责语义召回，BM25 (Elasticsearch) 负责关键词精确匹配，图谱检索 (Neo4j) 负责实体关系和多跳推理。三路各自召回后，用 RRF (倒数排名融合) 做融合去重，再用 Reranker 重排精排，兼顾语义和精确匹配。

面试官 检索质量怎么评估？总不能靠感觉吧。

我 我做了确定性的 Golden Eval：从 Recall、MRR、Citation Precision、拒答准确率这些维度回归验证，每次改代码都能量化对比。另外用 OpenTelemetry 做全链路 Trace，把「文档入库」和「问答」的每一步耗时串起来，定位慢在哪、错在哪。核心是「不只看能不能跑通，还要能量化度量」。

面试官 为什么用 Neo4j 而不是用关系库硬凑？

我 多跳关系遍历在图库里复杂度接近 $O(\text{深度})$ ，关系库要反复 JOIN 而且很难表达路径查询。Cypher 这种图查询语言天生适合「找关系、找路径」，这是图数据库和关系库的本质区别。

— 阶段 3 · 项目深挖 (AI Interview Agent) —

面试官 AI Interview Agent 为什么要用多智能体，而不是一个模型全干？

我 单一模型同时做「理解简历 + 提问 + 评分 + 写报告」这四件事，prompt 会非常混乱，质量也很难保证。我把流程拆成四个专业 Agent——简历解析、面试官、评分、报告，各司其职，每个都能独立优化、独立测试。

面试官 Orchestrator、SharedMemory、MessageBus 分别干什么？

我 Orchestrator 负责调度，决定先让谁跑、传什么数据、什么时候结束。SharedMemory 让多个 Agent 共享上下文，不用每个都从头理解一遍简历。MessageBus 是事件驱动的通信，Agent 之间通过消息解耦，不直接互相调用，方便扩展新角色。

面试官 评分是怎么做的？四个维度分别是什么？

我 从正确性、逻辑、深度、表达四个维度打分，通过 SSE 流式推给前端。正确性看答得对不对，逻辑看推理是否严密，深度看有没有超出表面的见解，表达看是不是结构清晰。按薄弱点自动追问，让练习更有针对性。

面试官 这个项目你怎么保证代码质量？

我 写了 149 个后端测试、114 个前端测试，覆盖核心流程。用 Docker Compose 容器化，GitHub Actions 做 CI/CD，每次提交自动跑测试。

— 阶段 4 · 技术八股 (RAG / LLM 方向) —

面试官 讲一下 RAG 的完整流程。

我 分三个阶段：索引阶段，文档解析 → 分块 → 向量化 → 入库；检索阶段，把 query 向量化，检索 top-k 相似块；生成阶段，把检索到的上下文和问题一起喂给 LLM，让它基于上下文作答，减少幻觉。我的 GraphMind 还加了异步入库和实体抽取写图谱。

面试官 为什么需要分块 (chunking) ?

我 LLM 上下文有限, embedding 也有长度限制, 整篇塞不下; 分块后按相关性只取相关的块。块太小丢上下文、太大噪声多, 要按语义边界分, 常用 512 到 1024 token。

面试官 混合检索为什么比纯向量好?

我 纯向量检索擅长语义相似, 但会漏掉精确关键词匹配; BM25 擅长关键词精确匹配但不懂语义。混合起来各取所长, 我的 GraphMind 还加了图谱检索做多跳推理。

面试官 RRF 是怎么融合多路排序结果的?

我 对每路检索的排序结果, 每个文档的分数等于 $\sum 1/(k + \text{rank})$, k 是常数通常取 60。它只依赖排名不依赖各路绝对分数, 所以不同尺度的分数能公平地融合。

面试官 怎么降低 LLM 的幻觉?

我 几个手段: 给足上下文 (RAG)、约束输出格式、让模型「不知道就说不知道」、温度调低、后置校验。结合我的产品, 鉴来助手被腾讯安全中心标记过「未经证实信息」, 本质就是 AI 生成内容的可信度问题, 所以我还加了免责声明、标注「AI 生成」。

面试官 上下文窗口太长放不下怎么办?

我 三种思路: 分块加检索 (RAG)、摘要压缩、滑动窗口。我的鉴来助手用的就是「渐进式摘要」——把几百章分层压缩成结构化记忆, 而不是全塞进上下文, 这是我最拿手的一个答案。

— 阶段 5 • 技术八股 (Python / FastAPI) —

面试官 解释一下 GIL, 多线程在 Python 里到底有没有用?

我 GIL 是 CPython 的机制, 同一时刻只有一个线程能执行字节码, 所以 CPU 密集任务多线程没法真正并行。但 IO 密集任务有用——线程在等 IO 时会释放 GIL。我的 FastAPI 后端大部分时间在等 DeepSeek 的 API, 所以用 async 协程做高并发, 比每请求开线程更轻。

面试官 FastAPI 里 def 和 async def 的接口有什么区别?

我 async def 在事件循环里跑, 适合 IO 密集; def 会被 FastAPI 丢到线程池, 不阻塞事件循环。坑是如果在 async def 里写阻塞操作, 会卡死整个事件循环, 所以调 DeepSeek 我用 httpx 的 async 客户端, 保持全链路异步。

面试官 SSE 和 WebSocket 的区别? 你为什么用 SSE?

我 SSE 是单向流 (服务器到客户端), 基于 HTTP, 简单还能自动重连; WebSocket 是双向、持久连接。我的面试系统是「提问 → 流式吐答案」的单向场景, SSE 足够, 比 WebSocket 简单。

面试官 JWT 是什么？和 Session 的区别？

我 JWT 是三段：header、payload、signature，签名用服务端密钥算 HMAC。无状态，服务端不存会话，适合水平扩展；缺点是没法主动失效，除非加黑名单。防伪造靠签名校验，改 payload 签名就对不上。密钥一定不能硬编码、不能泄露。

— 阶段 6 • 算法（口述思路） —

面试官 来道算法题热热身：给定一个数组和一个 target，找出两个数的下标，使它们的和等于 target。说下思路和复杂度。

我 用哈希表。遍历数组，对每个数 num 查 target - num 在不在表里，在就返回下标，不在就把 num 和它的下标存进表。时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

面试官 如果数组是有序的，能不能优化空间？

我 可以，用双指针。左右指针分别从两端走，和大了右指针左移，小了左指针右移。时间 $O(n)$ ，空间 $O(1)$ 。

— 阶段 7 • 行为面（STAR） —

面试官 讲一个你最有挑战、最难的时刻。

我 鉴来助手提交 Chrome 商店被审核打回那次。情境是扩展要上线但审核没过，任务是让它过审。我定位到权限声明太宽，把 host_permissions 收窄到单域名、逐条填写用途。结果是过审上线了双商店，也让我学到了「功能覆盖和审核风险的权衡」——不是功能越多越好。

面试官 你能实习多久？每周几天？

我 周内就能到岗，可以长期实习。这也是我投实习岗的定位，稳定性没问题。

— 阶段 8 • 反问环节 —

面试官 我这边问得差不多了，你有什么想问我的吗？

我 我了解两件事：一是这个岗位的实习生进来后，前一到三个月一般会做什么、怎么上手？二是团队现在用什么技术栈、最近在攻克什么技术问题？

我 另外想请教一下，您觉得能做好这个岗位的人，最重要的特质是什么？

— 阶段 9 • 结束 —

面试官 好的，今天先聊到这里，谢谢你的时间，我们会尽快反馈。

我 谢谢面试官，今天聊得很充分。如果后续需要我补充任何材料，随时联系我，期待能加入团队。再见。

第三部分 · 面试复盘卡

每场面试后打钩复盘（1~5 分）

维度	考察点	我的得分（1~5）
自我介绍	1 分钟内讲清定位和亮点	
项目深挖	能否讲清「为什么」和踩坑	
技术八股	答得是否流利、准确	
算法	是否做对、讲清复杂度	
行为面	STAR 是否结构化	
反问	是否问出有价值的问题	

复盘要点

- 这次被问了哪些题？哪些答得好、哪些卡壳？
- 卡壳的题：正确答案是什么？写下来，回填进《模拟面试全流程》手册对应章节。
- 面试官对我的项目哪个点最感兴趣？下次重点准备。
- 下一场要改的一个具体动作：_____

— 祝投递顺利 · 面试过关 —